FAIYAZ HUSSAIN H

API Data Retrieval and Storage: You are tasked with fetching data from an external REST API, storing it in a local SQLite database, and displaying the retrieved data. The API provides a list of books in JSON format with attributes like title, author, and publication year.

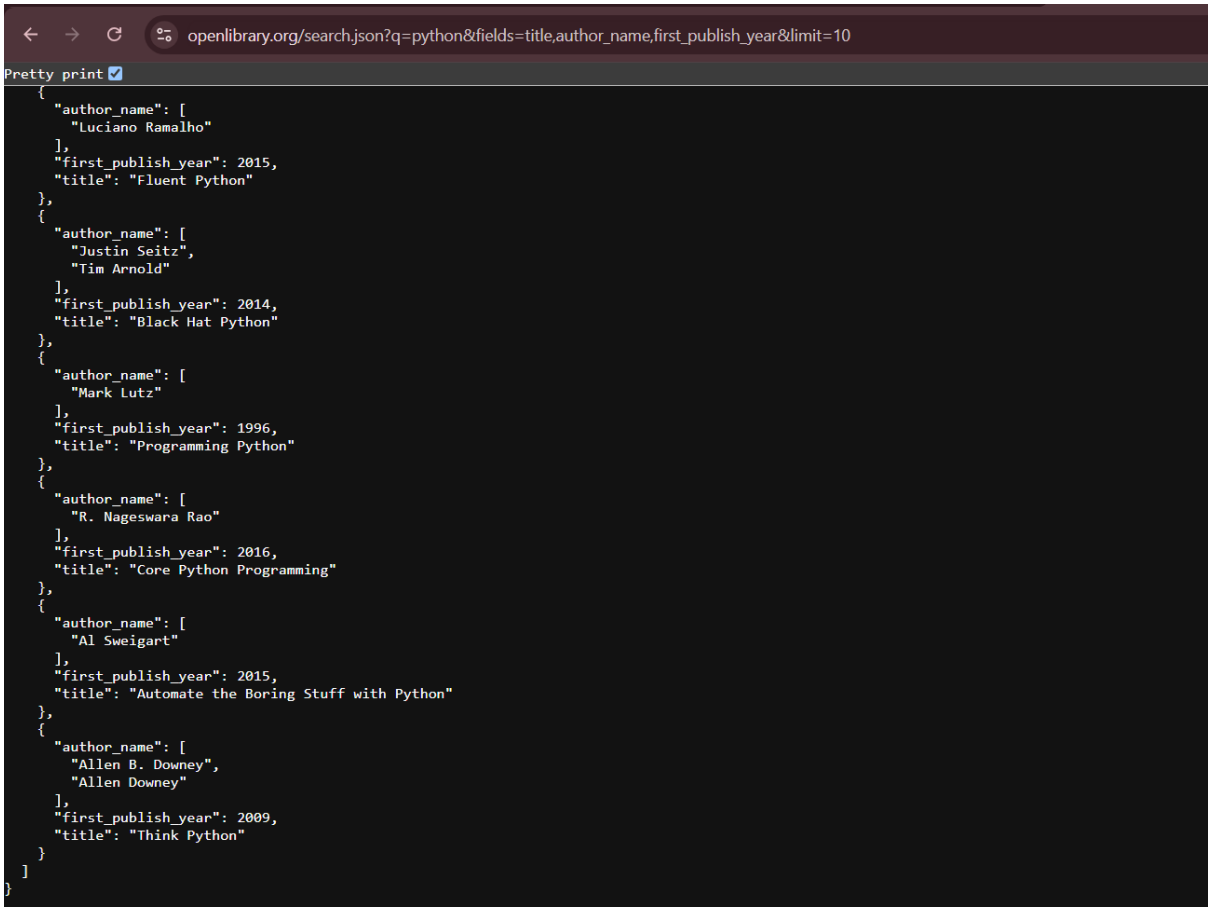
API Data Retrieval and Storage

What I built:

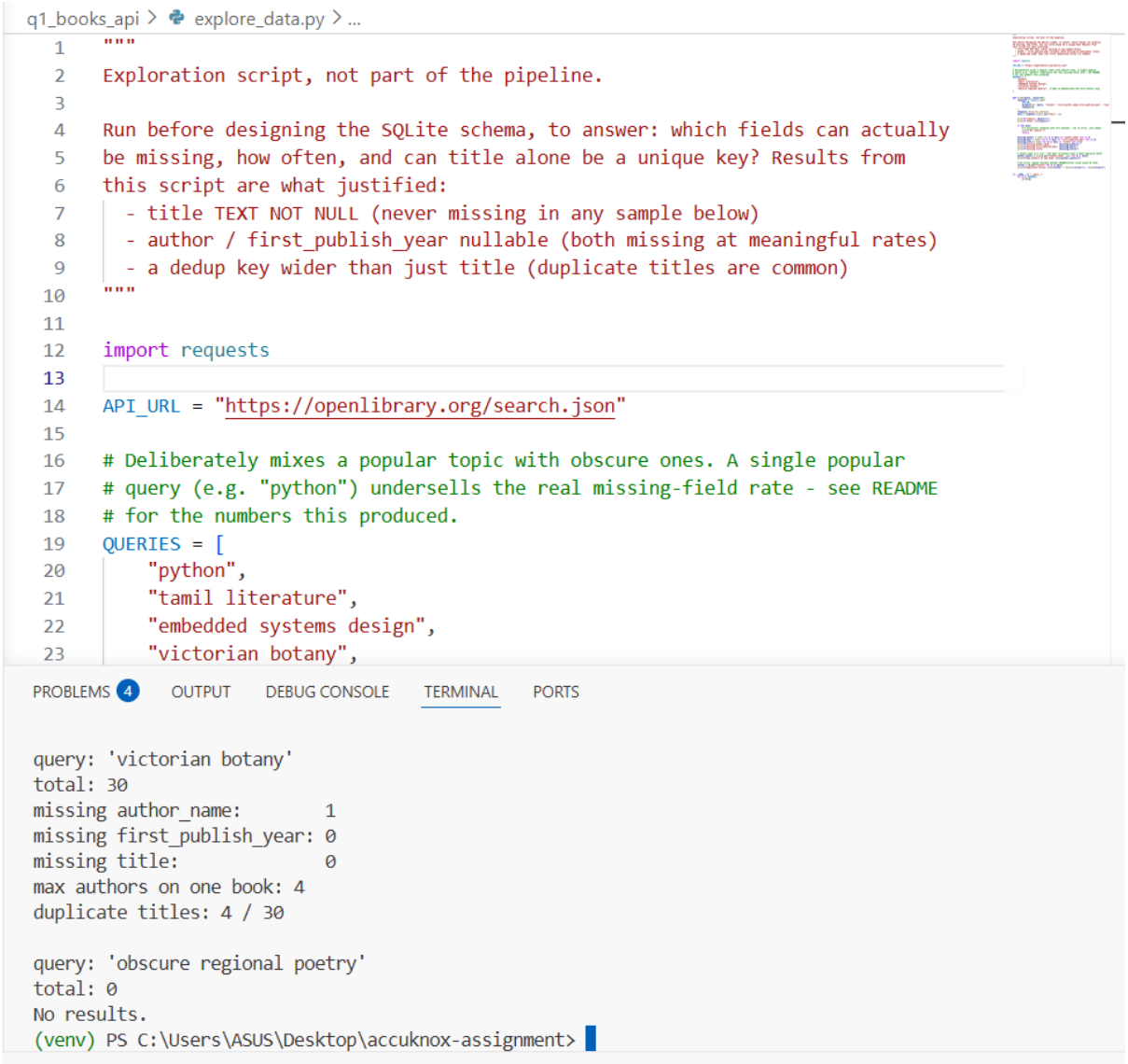
A single script, books_api.py, that fetches book records from the Open Library search API, stores them in a local SQLite database, and prints them back out. It's split into five functions — fetch_books, parse_books, init_db, save_books, display_books — tied together in main(), so each stage can fail and be handled on its own terms rather than everything sitting in one block.

API choice and assumptions:

I used Open Library's search.json endpoint. It needs no API key or authentication, returns plain JSON, and supports a fields= parameter so I could request only title, author_name, and first_publish_year instead of pulling the full record for every book.



Before writing the storage code I checked how reliable those fields actually are, across five different queries:



query	total	missing author	missing year	duplicate titles
python	100	1	2	not checked yet
tamil literature	100	6	1	9/100
embedded systems design	100	7	3	28/100
victorian botany	30	1	0	4/30
obscure regional poetry	0	—	—	—

Title was never missing in any sample, which is why the schema has title TEXT NOT NULL while author and year stay nullable. If I had only tested python, the missing-author

rate would have looked like 1%; on a less popular query it was seven times that. The duplicate-title counts are what stopped me using UNIQUE(title) on its own.

The zero-result query mattered too. Open Library returns HTTP 200 with an empty docs array, so "no matches" is a successful response containing nothing. My code treats that as a clean exit with a "no books found" message, separately from requests.RequestException, which means the network or server actually failed.

author_name comes back as a list, since books can have several authors. I join them into one comma-separated string. That's enough for storing and displaying, but it means you can't query for a single author without a LIKE scan. The proper fix would be a separate authors table with a junction table, which felt out of proportion for this task.

The duplicate problem

My first schema used a plain UNIQUE(title, author, first_publish_year) constraint. The first run against a fresh database parsed 100 rows and stored 99, which looked correct — the API had returned the same book twice and the constraint caught it.

The second run is where it broke. The row count went from 99 to 102 instead of staying at 99.

Two things were happening. Part of the growth was real: Open Library doesn't return an identical result set between calls, so some records were genuinely new. But querying the database directly showed the actual bug. SELECT * FROM books WHERE title = 'Think Like a Programmer, Python Edition' returned two rows, ids 94 and 194, with the same title, the same author, and first_publish_year NULL in both.

The reason is that in SQL, NULL is never equal to another NULL, including for uniqueness checks. So any book missing its year or author could be inserted an unlimited number of times without ever tripping the constraint. The constraint silently stopped constraining on exactly the rows I'd measured as most likely to have gaps.

I replaced the column-level UNIQUE with a separate index:

sql

```
CREATE UNIQUE INDEX idx_books_dedup
ON books (title COLLATE NOCASE, COALESCE(author, ''),
COALESCE(first_publish_year, -1))
```

COALESCE(x, sentinel) makes two NULLs compare as equal for the index, without the sentinel ever being written into the stored row — the table still holds NULL. I added COLLATE NOCASE on the title at the same time, because the live data had the same book under different capitalisation ("Core Python Programming" against "Core Python programming", "Starting Out with Python" against "Starting out with Python").

I verified both parts. Wiping the database and running twice gave 99 rows both times, and grouping on the same COALESCE expression with HAVING COUNT(*) > 1 returned nothing. For the case-insensitivity, I manually inserted LEARNING PYTHON against the existing Learning Python with the same author and year, and it was rejected with rowcount 0.

One thing that caught me out: CREATE INDEX IF NOT EXISTS matches on the index name, not its definition. A database created before the COLLATE NOCASE change keeps the old index indefinitely. I had to delete books.db before the fix did anything visible.

Other decisions

All inserts use parameterised ? placeholders rather than f-strings. Titles and author names are untrusted external text and can contain quotes.

save_books wraps executemany in with conn:, so the batch either commits fully or rolls back. No half-written table if something fails partway.

fetch_books sets timeout=10 and calls raise_for_status(). Without a timeout a stalled server hangs the process indefinitely, and without the status check a 404 or 500 body would be parsed as if it were book data.

I used try/finally around the connection rather than with sqlite3.connect(...), because that context manager commits the transaction but does not close the connection.

INSERT OR IGNORE makes reruns safe.

Known limitations

COLLATE NOCASE in SQLite only folds ASCII A–Z. My results include titles in Italian and Indonesian, so accented characters won't case-fold and those duplicates would still get through.

Author names aren't normalised at all. "J.K. Rowling" and "J. K. Rowling" are stored as different authors.

The COALESCE workaround is SQLite-specific. SQL Server treats NULLs as equal in a unique constraint, and PostgreSQL 15 added UNIQUE NULLS NOT DISTINCT to opt into that behaviour, so this would be written differently on another engine.

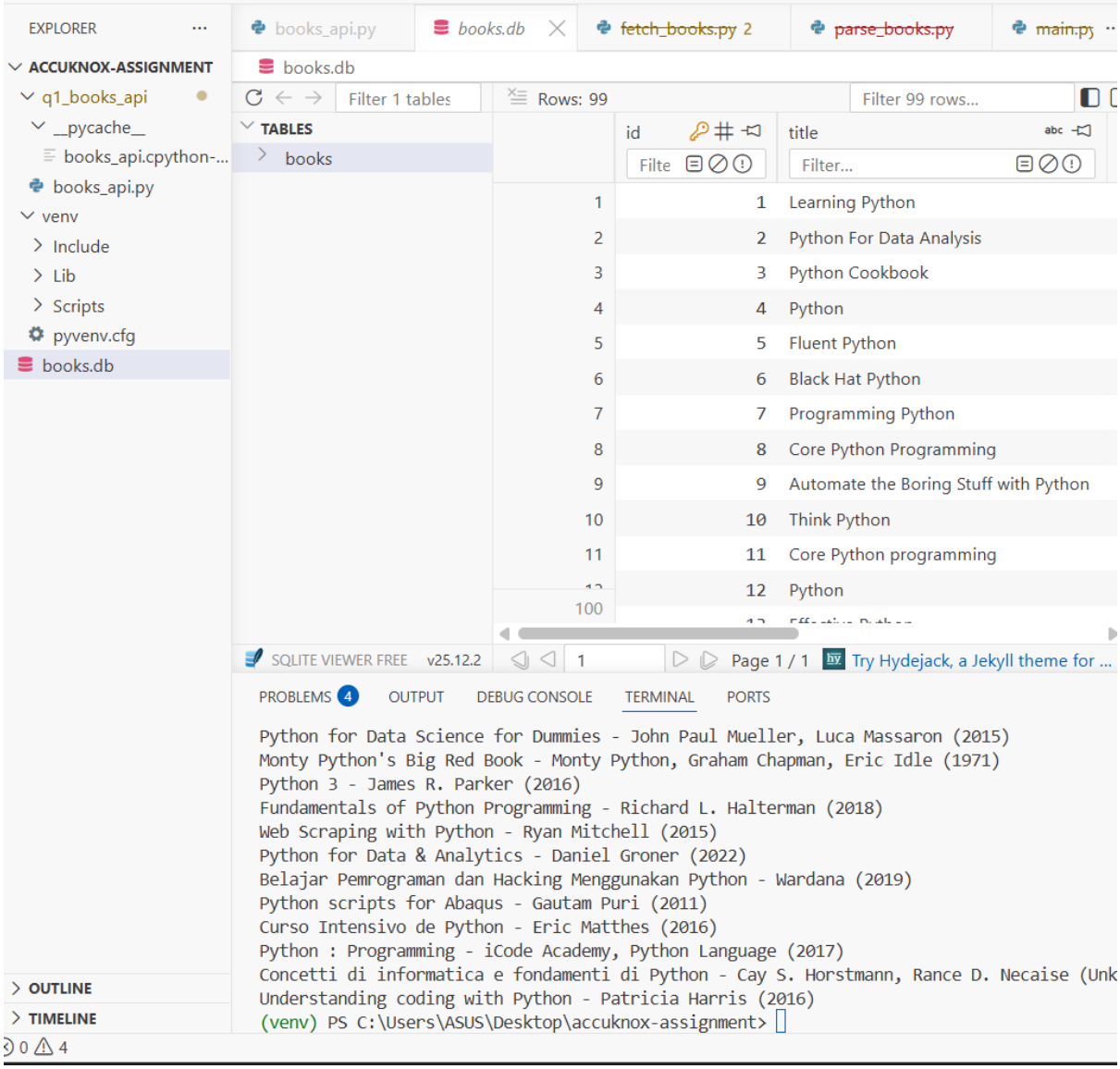
save_books returns the insert count but main() doesn't display it, so the output shows a before/after total rather than "X inserted, Y skipped".

The query is hardcoded to "python" rather than taking a CLI argument.

DB_PATH is relative to the working directory, so running the script from a different folder silently creates a different database.

Everything above was verified by hand against the live API. There's no test suite that would catch a regression.

Output:


EXPLORER
books_api.py
books.db
fetch_books.py 2
parse_books.py
main.py
ACCUKNOX-ASSIGNMENT
q1_books_api
__pycache__
books_api.cpython-...
books_api.py
venv
Include
Lib
Scripts
pyvenv.cfg
books.db
Filter 1 tables
Rows: 99
Filter 99 rows...
TABLES
books
id
title
Filte
Filter...
1 1 Learning Python
2 2 Python For Data Analysis
3 3 Python Cookbook
4 4 Python
5 5 Fluent Python
6 6 Black Hat Python
7 7 Programming Python
8 8 Core Python Programming
9 9 Automate the Boring Stuff with Python
10 10 Think Python
11 11 Core Python programming
12 12 Python
100
SQLITE VIEWER FREE v25.12.2
Page 1 / 1
Try Hydejack, a Jekyll theme for ...
PROBLEMS 4
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python for Data Science for Dummies - John Paul Mueller, Luca Massaron (2015)
Monty Python's Big Red Book - Monty Python, Graham Chapman, Eric Idle (1971)
Python 3 - James R. Parker (2016)
Fundamentals of Python Programming - Richard L. Halterman (2018)
Web Scraping with Python - Ryan Mitchell (2015)
Python for Data & Analytics - Daniel Groner (2022)
Belajar Pemrograman dan Hacking Menggunakan Python - Wardana (2019)
Python scripts for Abaqus - Gautam Puri (2011)
Curso Intensivo de Python - Eric Matthes (2016)
Python : Programming - iCode Academy, Python Language (2017)
Concetti di informatica e fondamenti di Python - Cay S. Horstmann, Rance D. Necaise (Unk
Understanding coding with Python - Patricia Harris (2016)
(venv) PS C:\Users\ASUS\Desktop\accuknox-assignment>
OUTLINE
TIMELINE
0 4